

Pipeline de **traduction de documents PDF avec préservation de la mise en page (WYSIWYG)** : on prend un PDF source, on comprend chaque page, on traduit le texte, puis on **recompose** la page traduite à l'identique (même fonds, mêmes figures, même typographie).

Architecture en une ligne

```
SOURCE (PDF / image / Office)
  → [moteur historique : ocr_server / translator / reconstructor]      (l'ancien, "legacy")
  → [pipeline moderne]  PAGEPRINT → PAGETRANSLATE → PAGERECONSTRUCT → PUBREADY  (QA)
```

Le projet contient **deux générations** qui coexistent :

1. **Le moteur historique** (fichiers à la racine) : monolithique, tout passe par `ocr_server.py` .
2. **Le pipeline moderne** (`pipelines/` , `pageprint/` , `pagetranslate/` , `pagereconstruct/` , `pubready/`) : un « clone propre » du moteur historique, découpé en unités testables, chacune avec un contrat de données explicite.

Le module que tu débugges (`pagereconstruct/`) est la **3^e tête** : c'est lui qui efface le texte source et redessine le texte traduit. C'est aussi la section la plus détaillée ci-dessous.

1. Le moteur historique (fichiers à la racine)

Ce sont les gros fichiers monolithiques, base de tout le savoir métier. Le pipeline moderne les « porte » progressivement.

Fichier	Rôle
<code>ocr_server.py</code> (5636 l.)	Serveur FastAPI + orchestrateur historique. Reçoit le PDF, convertit l'Office en PDF (LibreOffice headless), rend les pages, lance OCR, extraction, traduction et reconstruction. C'est le « cerveau » de l'ancienne génération que <code>pipelines/orchestrator.py</code> cherche à remplacer proprement.
<code>translator.py</code> (7398 l.)	DocumentTranslator : le traducteur métier complet (sélection sémantique, contexte page/document, terminologie, ton, protection des tokens non traduisibles, contrôle de longueur/overflow). Tout le reste (<code>pagetranslate</code> , <code>translation_engines</code>) gravite autour de lui.
<code>reconstructor.py</code> (8645 l.)	DocumentReconstructor : le reconstructeur historique. Géométrie des blocs, gabarits de ligne, placement, expansion de bbox, gestion des overlays. <code>pagereconstruct/</code> est la réécriture moderne et contractuelle de ce fichier.
<code>native_pdf_extractor.py</code>	Extraction native du PDF via PyMuPDF : texte, polices, couleurs, images, dessins vectoriels, zones non textuelles (bbox en points).
<code>structure_extractor.py</code>	<code>VisualAttributeExtractor</code> + <code>DocumentParser</code> + <code>LayoutV2Builder</code> : analyse visuelle d'une page et construction de la structure logique (blocs > lignes > phrases > spans).
<code>perfect_document_extractor.py</code>	Extracteur expérimental « haute fidélité », autonome, qui produit un modèle documentaire plus riche que le pipeline courant (ne le modifie pas).
<code>perfect_extraction_to_reconstruction.py</code>	Adaptateur expérimental : convertit la sortie du <code>perfect_document_extractor</code> vers l'entrée du <code>DocumentReconstructor</code> .
<code>perfect_reconstructor.py</code>	Sous-classe expérimentale du reconstructeur, change seulement le dispatch pour les pages produites par l'extracteur « parfait ».
<code>final_page_compiler.py</code> (1411 l.)	<code>ContinuousFinalPageCompiler</code> : compile une page finale continue (items texte, formules, opérations de dessin).

Compréhension / classification de page

Fichier	Rôle
<code>layout_descriptor.py</code> (2480 l.)	<code>LayoutDescriptorBuilder</code> : décrit la mise en page d'une page (colonnes, zones, structure). Version historique.
<code>layout_descriptor_v3.py</code> (1476 l.)	Version 3 du descripteur de layout.
<code>layout_compiler.py</code>	Compile des placements de layout (<code>LayoutPlacement</code>), calculs de chevauchement/colonnes.
<code>layout_optimizer.py</code>	<code>LayoutOptimizer</code> : optimisation de placement.
<code>layout_ai_enricher.py</code> (646 l.)	Enrichissement de la mise en page par modèle IA visuel (<code>LayoutAIEnricher</code>).
<code>visual_document_layout_model.py</code>	<code>VisualDocumentLayoutModel</code> : modèle de layout visuel (détection de structure par vision).
<code>page_case_classifier.py</code> (854 l.)	<code>PageCaseClassifier</code> : classe le « cas » d'une page (couverture, table des matières, index, corps, figure...).
<code>page_case_classifier_v2.py</code>	Version 2 du classifieur de cas de page.
<code>page_family_registry.py</code>	Registre des « familles » de pages et de leurs configs.
<code>page_profile_registry.py</code>	Registre des profils de page.
<code>page_policy_matrix.py</code> (510 l.)	<code>PagePolicyMatrix</code> : matrice de politiques (que faire selon le type de page/bloc).
<code>context_classifier.py</code>	<code>ContextClassifier</code> : classe le contexte (domaine, type de contenu).
<code>block_typology.py</code>	<code>classify_block_typology</code> : typologie d'un bloc (titre, paragraphe, liste...).
<code>special_region_detector.py</code> (1175 l.)	Détecte les zones spéciales (formules, tables, code, images, logos) à protéger.

Relations, géométrie, règles

Fichier	Rôle
<code>element_relations.py</code>	Enrichit les relations entre éléments (ordre de lecture, paires, confiance).
<code>element_relations_ai.py</code>	<code>ElementRelationsAIEnricher</code> : même chose, version IA.

<code>element_rulesets.py</code> (843 l.)	Construit des jeux de règles par phrase/bloc (annotation TOC, etc.).
<code>relative_geometry.py</code>	Enrichit la géométrie relative de la page (features, taille papier, direction de lecture).
<code>positioning_policy.py</code>	Politique de positionnement par bloc (scores sémantiques, alignement).
<code>logical_block_rebuilder.py</code>	Reconstruit les blocs logiques à partir de la géométrie.
<code>coverage_validator.py</code> (833 l.)	Valdateur de couverture : vérifie que tout le texte source est bien couvert (traduit/préservé/exclu) dans le rendu. Pièce centrale de la garantie « aucun texte perdu ».
<code>page_extraction_postprocessors.py</code> (1242 l.)	Post-traitements après extraction (fusion de bbox, tri des lignes, nettoyage).

Texte, style, traduction (historique)

Fichier	Rôle
<code>text_composer.py</code>	TextComposer : composition de texte (wrap, options de mise en ligne).
<code>style_profiler.py</code>	Profilage de style (couleurs, contraste, luminance).
<code>style_tone_classifier.py</code>	StyleToneClassifier : classe le ton/style du document.
<code>font_resolver.py</code>	FontResolver : résolution des polices (polices embarquées, répertoires système).
<code>terminology_manager.py</code>	TerminologyManager : gestion de la terminologie/glossaire.
<code>translation_memory.py</code>	TranslationMemory (historique).
<code>translation_validator.py</code>	TranslationValidator : validation de traduction.
<code>llm_semantic_corrector.py</code> (1411 l.)	Post-processeur LLM léger : corrige les segmentations sémantiques ambiguës (titres implicites, regroupement de légendes...).

Suppression de texte / fonds

Fichier	Rôle
<code>background_inpainter.py</code>	BackgroundInpainter : efface le texte source de l'image pour produire un fond propre (inpainting).
<code>remove_text_generic.py</code>	Suppression de texte générique (masque EAST + inpainting OpenCV).
<code>remove_text_dbnet_opencv.py</code>	Suppression de texte via DBNet + OpenCV (modèle de détection de texte).
<code>text_removal_strategy.py</code>	TextRemovalStrategy : choisit la stratégie de suppression.

Sortie / QA / divers racine

Fichier	Rôle
<code>html_exporter.py</code>	HtmlStyleExporter : export HTML stylé du document.
<code>visual_compare.py</code>	Comparaison visuelle (signatures de hauteur/gap/alignement) original vs reconstruit.
<code>publication_qa.py</code> (736 l.)	QA publication (version historique, comparaison à la source).
<code>render_quality_feedback.py</code>	Construit un rapport de 2 ^e passe sur les défauts de rendu.
<code>evaluate_layout_quality.py</code>	Évalue la qualité de layout (ratio de chevauchement).
<code>runtime_config.py</code>	Configure l'environnement « agentless » (variables d'env par défaut). Importé en tout premier par <code>translator.py</code> .

2. pipelines/ — l'orchestrateur moderne (clone propre d' ocr_server.py)

Chaîne : `SourceLoader` → `PageRenderer` → `RawExtractors` → `PageUnderstanding` → `PAGEPRINT`.

Fichier	Rôle
<code>pipelines/_init_.py</code>	Décrit l'architecture cible de l'orchestrateur.
<code>pipelines/orchestrator.py</code>	PipelineOrchestrator : enchaîne les unités page par page et produit l' <code>INPUT_DATA</code> canonique. C'est le remplaçant propre du monolithe <code>ocr_server.py</code> .
<code>pipelines/source_loader.py</code>	Unité 1 : lit PDF/image/Office, convertit l'Office en PDF, calcule le hash et les métadonnées source.
<code>pipelines/page_renderer.py</code>	Unité 2 : rend la page en image (DPI, scale px+pt). Le pixel ne sert qu'à l'OCR/masques/debug ; le modèle reste en points.
<code>pipelines/raw_extractors.py</code>	Unité 3 : extraction brute multi-sources (NativePDF + OCR optionnel).
<code>pipelines/page_understanding.py</code>	Unité 4 : compréhension de page (rôle, famille, type de doc, style, relations) via <code>LayoutV2Builder</code> , avant PAGEPRINT.
<code>pipelines/pageprint_stage.py</code>	Unité 5 : lance la tête PAGEPRINT (normalise → résout conflits → units → regions → graph → polices → constraints → quality).
<code>pipelines/ocr_router.py</code>	Décide si l'OCR tourne en pleine page, sur des régions visuelles, ou pas du tout.
<code>pipelines/background_cleaner.py</code>	Génère le fond « text-removed » (inpainting du texte source).
<code>pipelines/background_cover.py</code>	Fond déterministe « couvre-texte » : chaque ligne source à redessiner est recouverte d'une couleur estimée localement.
<code>pipelines/document_context.py</code>	Contexte document : en-têtes/pieds de page répétés, marques.

3. [pageprint/](#) — PAGEPRINT (1^{re} tête : compréhension canonique)

Transforme une page source en `INPUT_DATA` : représentation **canonique, normalisée, enrichie, vérifiable**, consommée par tous les modules aval. C'est la **source de vérité**.

Cœur

Fichier	Rôle
<code>pageprint/__init__.py</code>	Description de la tête PAGEPRINT.
<code>pageprint/builder.py</code> (1029 l.)	Construit l' <code>INPUT_DATA</code> (l'empreinte canonique de la page). Orchestrateur de tous les sous-modules ci-dessous.
<code>pageprint/schema.py</code>	Contrat canonique <code>pageprint.input.v1</code> (DTOs, constantes).
<code>pageprint/normalizer.py</code>	Normalise unités/points/styles. Règle d'or : modèle interne en points , pixels seulement aux frontières image/OCR.
<code>pageprint/unit_factory.py</code> (857 l.)	Transforme blocs/lignes/phrases/spans en unités canoniques (liste plate, chaque unité porte ses champs requis).
<code>pageprint/region_index.py</code>	Indexe régions/zones/objets (images, dessins, tables, formules, code...) et calcule les appartenances.
<code>pageprint/graph_builder.py</code>	Construit le graphe documentaire (page ⊃ region ⊃ block ⊃ line ⊃ phrase ⊃ span).
<code>pageprint/graph_query.py</code>	Helpers de requête graphe/rôle, partagés avec le fallback PAGETRANSLATE.
<code>pageprint/role_resolver.py</code> (508 l.)	Résout le rôle documentaire de chaque unité (titre, corps, légende, formule...).
<code>pageprint/semantic_builder.py</code> (516 l.)	Construit les unités sémantiques à partir des rôles et structures logiques.

Décision, preuve, qualité

Fichier	Rôle
<code>pageprint/evidence_resolver.py</code>	Résout les conflits entre sources (native_pdf / ocr / layout_ai / detector / heuristics / LLM) : qui gagne.
<code>pageprint/evidence/__init__.py</code>	Sous-paquet preuve.
<code>pageprint/evidence/claim_model.py</code>	DTO de « claim » (revendication d'une source).
<code>pageprint/evidence/collector.py</code>	Collecte les revendications concurrentes depuis les unités normalisées.
<code>pageprint/evidence/resolver.py</code>	Résout ces revendications en compréhension d'unité.
<code>pageprint/provenance.py</code>	Mémoire de décision : pourquoi tel bloc est une légende, pourquoi telle phrase n'est pas traduite... (pipeline rejouable).
<code>pageprint/quality_assessor.py</code>	Score de confiance, risques, fragments suspects, unités faibles, risques aval .
<code>pageprint/policy_compiler.py</code>	Compile les policies (ordres exécutables aval : skip_translation, background_only...).
<code>pageprint/preservation_compiler.py</code>	Compile les modes de préservation depuis rôles/preuve.
<code>pageprint/constraint_compiler.py</code>	Compile les contraintes WYSIWYG par unité (preserve_bbox, allow_reflow, allow_wrap, preserve_alignment...).
<code>pageprint/view_compiler.py</code>	Compile les plans d'exécution aval (les « vues » consommées par pagetranslate/pagereconstruct).
<code>pageprint/validators.py</code>	Valide que l' <code>INPUT_DATA</code> est exploitable (bbox en points, unit_id uniques, parents existants...).
<code>pageprint/functional_validator.py</code>	Validation fonctionnelle au-delà du schéma.
<code>pageprint/serializers.py</code>	Sérialisation JSON de l' <code>INPUT_DATA</code> .
<code>pageprint/text_postprocessors.py</code>	Nettoyage pré-translation (dé-césure intra-texte et inter-segments).

Détection de régions

Fichier	Rôle
<code>pageprint/detection/__init__.py</code>	Phase interne de détection de régions.
<code>pageprint/detection/builder.py</code>	Orchestre la détection (détecteur hybride legacy + fusion avec régions amont).
<code>pageprint/detection/schema.py</code>	Constantes de schéma de détection.

Constructeurs de structures logiques ([pageprint/structure_builders/](#))

Chacun construit un type d'unité logique sans dupliquer bloc/ligne/phrase :

Fichier	Construit...
<code>__init__.py</code> / <code>common.py</code>	Paquet + helpers partagés.
<code>heading_builder.py</code>	Les titres.
<code>body_paragraph_builder.py</code>	Les paragraphes de corps.
<code>list_builder.py</code>	Les items de liste.

table_builder.py (243 l.)	Les tables (détection de lignes/grille).
toc_builder.py	Les entrées de table des matières.
index_builder.py	Les entrées d'index.
caption_builder.py	Les légendes.
figure_builder.py	Les figures (+ légendes proches, labels de diagramme).
formula_builder.py	Les unités de formule.
code_builder.py	Les blocs de code.
author_bio_builder.py	Les bios d'auteur.
publisher_mark_builder.py	Les marques éditeur / artefacts (à exclure de la traduction).

4. [pagetranslate/](#) — PAGETRANSLATE (2^e tête : traduction)

Consomme l' `INPUT_DATA` de PAGEPRINT, sélectionne les unités traduisibles, ajoute le contexte, traduit, puis réinjecte. Produit `translated_input_data`.

Fichier	Rôle
pagetranslate/__init__.py / README.md	Description de la tête.
pagetranslate/builder.py (486 l.)	Pipeline métier : sélection → protection → traduction → contrôle qualité → projection.
pagetranslate/schema.py	Constantes et DTOs du contrat de traduction.
pagetranslate/selector.py (480 l.)	Sélection sémantique des unités traduisibles (<code>semantic_phrase</code> > <code>group</code> > <code>phrase</code> > <code>line</code> > <code>block</code>), exclut <code>word/char</code> .
pagetranslate/coalescer.py	Fusionne les unités visuelles en phrases sémantiques quand nécessaire.
pagetranslate/sentence_boundary.py	Frontières de phrase (début/fin, abréviations, multi-ligne, hard break vs soft wrap).
pagetranslate/protection.py	Placeholderise puis restaure les tokens intouchables (URL, DOI, emails, nombres, unités, formules, chemins, références).
pagetranslate/technical_protection.py	Protège les tokens techniques dans code/table (<code>None/True/False</code> , <code>Conv2D</code> , shapes, SQL, chemins...).
pagetranslate/context_builder.py	Construit le contexte (avant/après, page, domaine, style, ton, contraintes WYSIWYG).
pagetranslate/terminology.py	Support terminologie/glossaire.
pagetranslate/translator_bridge.py	Appel propre à <code>DocumentTranslator</code> , avec retry si sortie vide/identique.
pagetranslate/quality.py (239 l.)	Contrôles qualité + <code>needs_review</code> (expansion, tokens protégés, nombres, unités).
pagetranslate/functional_validator.py	Vérifications fonctionnelles de la sortie.
pagetranslate/projection.py (399 l.)	Réinjecte les traductions dans l' <code>INPUT_DATA</code> + vue compatible reconstruction.
pagetranslate/text_survival.py (496 l.)	Invariant dur : toute ligne source visible DOIT avoir un chemin de sortie (traduite OU préservée OU exclue avec raison). Privilégie la présence du texte à la beauté du layout.
pagetranslate/translation_plan_reader.py	Lit <code>views.translation_plan</code> de PAGEPRINT.
pagetranslate/text_utils.py	Petits helpers texte partagés.

5. [pagereconstruct/](#) — PAGERECONSTRUCT (3^e tête : recomposition) ☆

« **Not a text drawer — a document recomposition solver.** » On ne redessine pas du texte : on respecte un plan de couches, protections, patches, styles et contraintes.

C'est **le module que tu débugges**. Les deux bugs connus (texte source non effacé + bbox qui écrase le texte sur la 1^{re} ligne) vivent ici, surtout dans `layout_box_resolver.py`, `background_resolver.py`, `patch_planner.py` et les backends.

Pipeline interne

```
translated_input_data
→ input_adapter          (4 vues)
→ plan_compiler           (fusion, anti double-rendu)
  layout_box_resolver / style_resolver / background_resolver / patch_planner
→ PageRenderPlan
→ renderer_dispatcher + renderers/
→ backends/ (pdf_vector | raster_debug)
→ validator / quality / visual_qa → status ok|review|ko
```

Entrée & compilation du plan

Fichier	Rôle
pagereconstruct/__init__.py / README.md	Description + philosophie de la tête.

pagereconstruct/schema.py	Contrats de données : le PageRenderPlan intermédiaire (couches + protections + ids consumed/excluded).
pagereconstruct/input_adapter.py	Normalise translated_input_data en 4 plans : reconstruction_units (texte traduit), reconstruction_plan, preservation_plan, exclusion_plan.
pagereconstruct/plan_compiler.py (779 l.)	Compile le PageRenderPlan : fusionne les 4 vues + géométrie + couches visuelles, calcule consumed/excluded (anti double-rendu). Pièce maîtresse.

Résolveurs (où vivent tes bugs) 🐛

Fichier	Rôle
pagereconstruct/layout_box_resolver.py	🔍 Résout layout/patch/anchor bbox par rôle . Corrige précisément le bug que tu décris : un paragraphe multi-ligne est effacé sur toute sa surface mais le texte est redessiné dans la bbox de la 1^{re} ligne . Ici, le texte de flux doit se mettre en page dans layout_bbox = bloc logique complet. A regarder en priorité .
pagereconstruct/background_resolver.py	🔍 Décide le fond + le risque de fuite du texte source : clean_background > source_background (debug) > blank_degraded . Si le texte source n'est pas effacé, c'est souvent ici ou dans le patch_planner.
pagereconstruct/patch_planner.py	🔍 Planifie les zones de patch (cleanup) au-dessus du texte source. Chaque unité traduite reçoit un patch sur sa zone source ; refusé sur région protégée.
pagereconstruct/style_resolver.py	Résout un style typographique exploitable pour CHAQUE unité (aucune unité ne doit arriver au backend avec style = {}).
pagereconstruct/font_resolver_bridge.py	Déduit la vraie classe de police depuis les noms de subset PDF mutilés (serif/sans/mono).
pagereconstruct/font_size_sanitizer.py	🔍 Répare les tailles de police absurdes (le démo montrait ~4.78pt pour du corps qui fait 9-11pt — l'extracteur sort parfois une métrique de glyphe, pas la taille réelle). Lié à ton 2 ^e bug (texte illisible).

Contrats

Fichier	Rôle
pagereconstruct/final_contract.py	FinalReconstructionContract : source unique et non ambiguë du rendu (fusion pageprint + pagetranslate + savoir legacy).
pagereconstruct/block_contract.py	Contrat complet d'un bloc à rendre (géométrie + style + politique + préservation + qualité).
pagereconstruct/layout_contract.py	Géométrie + libertés de déplacement (gabarits de ligne, padding, keep_with).
pagereconstruct/style_contract.py	Typographie figée d'une unité (jamais inventée par le rendu ; provenance extracted/inferred/repaired).
pagereconstruct/background_contract.py	Quel fond + risque de fuite (fond non nettoyé = bloquant en publication).
pagereconstruct/preservation_contract.py	Objets gardés tels quels (pixels) + texte exact (overlays/underlays).
pagereconstruct/object_contract.py	Identité/politique d'un objet de page.
pagereconstruct/quality_contract.py	Seuils publication-ready (gates durs : must_render / no clip / no overlap).
pagereconstruct/page_level_contracts.py	Contrats des objets « page-level » (politique de traduction/préservation/ancrage explicite).

Composition (avant rendu)

Fichier	Rôle
pagereconstruct/composition/block_planner.py	Place les GRANDS blocs avant l'intra-bloc (solveur multi-blocs).
pagereconstruct/composition/intrablock_composer.py	Compose le contenu DANS un bloc (lignes, runs, objets inline) — le renderer ne compose plus.
pagereconstruct/composition/typography_planner.py	Produit le TypographyPlan (échelle de page, em estimé) avant le rendu.
pagereconstruct/composition/special_zone_preserver.py	Classe les objets non traduisibles en 4 niveaux (block/inline/page/background).

Solveurs de placement

Fichier	Rôle
pagereconstruct/candidate_engine.py	Génère et score des candidats de rendu (6 scores : text_fit, style, position, collision, readability, preservation).
pagereconstruct/placement_solver.py	Choisit le meilleur candidat sans collision , avant le rendu (normal → shrink → interligne compact → shift).
pagereconstruct/collision_detector.py	Détecte les chevauchements sur la géométrie réellement rendue (texte/texte, texte/protégé).
pagereconstruct/block_expansion_solver.py	Étend la bbox d'un bloc + reflow des voisins après traduction (le texte traduit est souvent plus long).
pagereconstruct/multiblock_layout_solver.py	Résout les collisions de page dense en optimisant des groupes de blocs dans les régions de flux.
pagereconstruct/layout_reflow_solver.py	Solveur d'espacement/reflow conservateur (CPU-only) qui préserve les garanties acquises.

Rendu : ops, dispatch, renderers, backends

Fichier	Rôle
pagereconstruct/ops.py	Opérations de rendu explicites — le backend les exécute, il n'improvise pas.
pagereconstruct/render_ops.py	RenderOps : instructions de dessin plates et résolues (dispatch + mesure ont lieu UNE fois ici, en espace PT).
pagereconstruct/renderer_dispatcher.py	Mappe rôle → renderer concret. Un rôle inconnu ne devient JAMAIS le renderer paragraphe .
pagereconstruct/text_measure.py	Mesure de texte canonique (largeur de ligne, wrap, ajustement) — une seule porte de mesure. Pure, pas de dessin.
pagereconstruct/render_backend.py	Backend raster (PIL) « style-aware » : efface les zones source via patches, puis dessine le texte traduit.
pagereconstruct/backends/pdf_vector.py	Backend PDF vectoriel (PyMuPDF) — sortie finale V1 . Fond + rectangles patch + texte via insert_textbox .

<code>pagereconstruct/backends/raster_debug.py</code>	Backend PNG de debug (overlays, contact sheets) — pas la sortie WYSIWYG.
<code>pagereconstruct/renderers/base.py</code>	Layout/measure partagé + renderer de base (mesure d'abord → peint depuis la mesure).
<code>pagereconstruct/renderers/paragraph.py</code>	Paragraphes + items de liste.
<code>pagereconstruct/renderers/heading.py</code>	Titres.
<code>pagereconstruct/renderers/code.py</code>	Code.
<code>pagereconstruct/renderers/formula.py</code>	Formules.
<code>pagereconstruct/renderers/index.py</code>	Index.
<code>pagereconstruct/renderers/bibliography.py</code>	Bibliographie.
<code>pagereconstruct/renderers/table_cell.py</code>	Cellules de table.
<code>pagereconstruct/renderers/anchored_label.py</code>	Labels ancrés (+ variante review pour rôle inconnu).
<code>pagereconstruct/renderers/preservation.py</code>	Objets préservés (pixels).

Préservation, protection, traçabilité

Fichier	Rôle
<code>pagereconstruct/overlay_manager.py</code>	Gère underlays (sous le texte : figures, fonds, formules) vs overlays (au-dessus : numéros de page, labels), attribue le <code>z_index</code> .
<code>pagereconstruct/protected_region_index.py</code>	Index spatial des régions protégées (zones où le renderer n'écrit pas et le patch n'efface pas).
<code>pagereconstruct/text_removal_ledger.py</code>	Registre de suppression : une ligne par texte source remplacé — la suppression était-elle attendue et vérifiée. Garantit qu'aucun texte source ne fuit.
<code>pagereconstruct/source_text_lifecycle_ledger.py</code>	Cycle de vie par unité (décision traduction → entrée reconstruction → op de rendu → statut visuel).
<code>pagereconstruct/source_text_leak_detector.py</code>	🕒 Détecte le texte source résiduel : une zone de patch quasi identique source/reconstruit = ancien texte non effacé (= leak).

Validation & QA

Fichier	Rôle
<code>pagereconstruct/validator.py</code>	Les findings gouvernent le statut <code>ok</code> <code>review</code> <code>ko</code> .
<code>pagereconstruct/quality.py</code>	Métriques de qualité dérivées du <code>PageRenderPlan</code> .
<code>pagereconstruct/visual_qa.py</code>	Score sur les 6 critères publication (présence texte/non-texte, overlap, position, typo).
<code>pagereconstruct/invariant_guard.py</code>	Verrouille les invariants à ne jamais régresser (toute ligne visible a un owner ; fond propre vérifié ; pas de fallback silencieux).
<code>pagereconstruct/style_similarity.py</code>	Score de fidélité du style résolu vs source (publication ≥ 0.95 ; < 0.85 = ko).

Auto-correction & ponts legacy

Fichier	Rôle
<code>pagereconstruct/autocorrect/correction_loop.py</code>	La validation corrige, pas seulement constate : compile → audit → corrige (knob) → recompile → garde le meilleur score.
<code>pagereconstruct/autocorrect/correction_plan.py</code>	Traduit les findings d'audit en actions de correction.
<code>pagereconstruct/autocorrect/retry_policy.py</code>	Politique de réessai de la boucle.
<code>pagereconstruct/legacy_contract_bridge.py</code>	Rend le savoir de l'ancien moteur (process_page / reconstructor) disponible dans le contrat moderne.
<code>pagereconstruct/legacy/function_registry.py</code>	Registre de migration des fonctions legacy (KEEP_AS_IS / ADAPT / WRAP / MERGE / DROP / REPLACE_TESTED).
<code>pagereconstruct/integration_adapter.py</code>	Adaptateur optionnel pour 2 modules externes « 95%-unlock ».
<code>pagereconstruct/ocr_typography_engine.py</code>	Améliore la typographie OCR (estimation font-em stable, échelle de style).
<code>pagereconstruct/templates/book_figure_page.py</code>	Template pour page de livre avec figure + légende.
<code>pagereconstruct/errors.py</code>	Exceptions du module.

6. pubready/ — QA publication-ready (granulaire et explicable)

Hiérarchie d'audit : page → blocs → phrases → dimensions , chaque dimension comparée à l'origine PAGEPRINT. Une page ko / texte manquant / leak / objet détruit bloque le document .	
Fichier	Rôle
<code>pubready/evaluator.py</code>	Entrée unique : normalise l'input traduit puis lance <code>evaluate_page</code> (additif, ne touche pas le rendu).
<code>pubready/page_auditor.py</code>	Évaluateur de page : lance les audits granulaires + réutilise la QA visuelle, combine selon les gates.
<code>pubready/document_auditor.py</code>	Consolidation document (score ≠ moyenne simple).

pubready/gates.py	Seuils stricts par étape + hard blockers + règles de décision page/document.
pubready/schema.py	Schémas de score granulaires/explicables.
pubready/evidence.py	Accès aux données d'origine PAGEPRINT pour comparer reconstruit vs source.
pubready/reports.py	Export des rapports (JSON + Markdown).
pubready/stages/typography_audit.py	Audit typo granulaire (classe police, taille, couleur, gras, italique, alignement).
pubready/stages/translation_audit.py	Audit traduction par bloc (présence, non-vide, troncature, tokens protégés, code/formule non traduits).
pubready/stages/position_audit.py	Audit position : la bbox de rendu reste ancrée sur la source, sans dérive ni collision.
pubready/stages/background_audit.py	Audit fond propre (trame nettoyée + registre complet + patches non destructeurs).
pubready/stages/preservation_audit.py	Audit préservation (formule/code/image/table/logo protégés et non écrasés).
pubready/stages/intrablock_audit.py	Audit composition intra-bloc (aucun mot perdu, pas d'overflow/clipping).
pubready/stages/contract_audit.py	Audit du FinalReconstructionContract (chaque bloc a layout/style/renderer).
pubready/stages/render_ops_audit.py	Audit RenderOps (BackgroundOp présent, TextOps couvrants, parité PDF/PNG).
pubready/stages/pageprint_audit.py	Audit PAGEPRINT (fournit-il assez pour reconstruire ?).
pubready/stages/visual_image_audit.py	🔍 Regarde réellement l'image finale (cv2) : zones protégées ~identiques (sinon objet détruit), zones de texte remplacé ayant changé (sinon leak = ancien texte visible).

7. translation_engines/ — moteurs de traduction enfichables

Fichier	Rôle
__init__.py	Moteurs pour les essais contrôlés PAGEPRINT/PAGETRANSLATE.
base.py	TranslationEngine + TranslationResult (interface commune).
request.py	TranslationRequest (DTO d'entrée).
factory.py	create_translation_engine (fabrique).
model_registry.py	Registre de modèles (sélection NLLB / M2M100 / Marian / Opus selon la paire de langues).
ct2_engine.py	Moteur CTranslate2 (encode source, préfixe cible, décode hypothèses).
local_model_engine.py	Moteur de modèle local.
external_model_engine.py	Moteur de modèle externe (API).
rule_engine.py	Moteur à règles.
mock_engine.py	Moteurs factices (Mock/Prefix/Echo) pour tests.
translation_memory.py	Mémoire de traduction validée (un hit fait sauter le modèle).
placeholder_policy.py	Choix du style de placeholder + construction/variantes.
profile_store.py	Charge translation_profiles.json / style_tone_profiles.json (seuils qualité, post-édition, glossaire, ton).
engine_health.py	Rapport de santé du moteur.

8. pipeline_agents/ — agents LLM open-source par étape (P1-P7)

Chaque agent encapsule un LLM local (Phi-3.5 Mini, Qwen2.5, GGUF...) derrière une interface identique, provider-agnostique. Chacun a un équivalent heuristique sans LLM.

Fichier	Rôle
__init__.py / base.py	Paquet + classes de base (interface commune indépendante du backend).
registry.py	Factory get_agent(...) provider-agnostique.
heuristics.py	Estimateurs déterministes (sans LLM) pour P1, P3, P5, P6.
p1_extraction.py	P1 — extraction & segmentation sémantique (titres, formules, légendes, frontières).
p2_structure.py	P2 — structuration hiérarchique (parent/enfant, sections, regroupement).
p3_layout.py	P3 — mode de layout (inline_reflow / preserve_line_breaks / preserve_paragraphs).
p4_translation.py	P4 — validation & post-édition de traduction (score, problèmes, version corrigée).
p4_qe_estimator.py	P4 QE — estimation qualité sans génération LLM (heuristique / CometKiwi...).
p5_render.py	P5 — stratégie de rendu (prose_reflow / label_stack...).
p6_background.py	P6 — audit du fond (artéfacts textuels résiduels, zones à re-peindre).
p7_publication_layout.py	P7 — mise en page finale publication-ready au niveau page.

9. `server/` — couche HTTP mince (clone moderne)

Fichier	Rôle
<code>server/_init_.py</code>	Le serveur ne contient plus l'intelligence : upload, routing, appel du pipeline, retour JSON.
<code>server/api.py</code> (391 l.)	API du clone : upload multi-fichiers, routing, appel de l'orchestrateur, persistance des résultats.
<code>server/static/index.html</code>	Front statique minimal.

10. `tools/` — outils internes (audit, démos, patches, Rule Studio)

Audits & essais

Fichier	Rôle
<code>tools/audit_text_survival.py</code>	Garantit que 100% du texte PAGEPRINT se retrouve dans le rendu (traduit/préservé/exclu avec raison).
<code>tools/audit_translation_selection.py</code>	Audit de la qualité de sélection de traduction (granularité choisie, ce qui est exclu/protégé).
<code>tools/run_functional_audit.py</code> / <code>run_batch_functional_audit.py</code>	Audit fonctionnel strict (rev_04) sur l' <code>INPUT_DATA</code> , unitaire ou en lot.
<code>tools/run_translation_trial.py</code> / <code>run_batch_translation_trial.py</code>	Essais de traduction contrôlés, unitaire ou en lot.
<code>tools/run_document_trial.py</code>	Extrait un PDF puis lance un essai de traduction contrôlé.
<code>tools/run_pageprint_pagetranslate_audit.py</code>	Audit PAGEPRINT→PAGETRANSLATE sur pages PDF aléatoires.
<code>tools/run_pipeline_full_demo.py</code>	Pipeline complet (PAGEPRINT→PAGETRANSLATE→PAGERECONSTRUCT) sur des pages, dump des artefacts.
<code>tools/compare_legacy_reconstruction.py</code>	Compare ancien/nouveau rendu + score pubready (heatmap de diff).
<code>tools/check_translation_engine.py</code> / <code>translate_text_smoke.py</code> / <code>test_placeholder_roundtrip.py</code>	Smoke-tests moteur/placeholders.
<code>tools/export_pipeline_inventory_xlsx.py</code>	Inventaire page par page en XLSX (un onglet par page).

Démos locales

Fichier	Rôle
<code>tools/local_demo_app.py</code>	App Tkinter locale (choisir PDF, pages, niveau de pipeline, lancer).
<code>tools/local_demo_runner.py</code>	Backend CLI de l'app ci-dessus.
<code>tools/demo_studio_backend.py</code>	Backend local (non-web) pour un « Demo Studio » Flutter (émet des événements).

Patches ponctuels (`tools/patch_*.py`)

Scripts qui **patchent du code** de façon ciblée (correctifs déterministes appliqués à des fichiers précis) : `patch_atomic_patch_planner` , `patch_atomic_text_builder` , `patch_atomic_text_projection` , `patch_background_cleaner_deterministic` , `patch_demo_backend_background_translation` , `patch_demo_studio_main` , `patch_plan_compiler_spacing_reflow` , `patch_projection_consume_translations` , `patch_text_survival_builder/projection/reader` , `patch_vsense_studio_main` . Ce sont des correctifs d'historique (utiles pour comprendre les bugs déjà traités, dont certains touchent ton sujet : `spacing_reflow` , `background_cleaner` , `text_survival`).

`tools/pipeline_dashboard/` — tableau de bord Streamlit

Fichier	Rôle
<code>app.py</code>	Dashboard dense et éditable (1 ligne = 1 élément granulaire) pour vérifier l'application des règles page par page.
<code>elements.py</code>	Construit les enregistrements plats (jointure PAGEPRINT × PAGETRANSLATE).
<code>ingest.py</code>	Ingestion d'un run d'audit en base SQLite + copie des assets.

`tools/rule_studio/` — console de gouvernance des règles

Outil complet qui **scanne le dépôt, extrait les règles dispersées, les classe par étape de pipeline, les audite, les simule et propose des patches tracés** (jamais d'écriture silencieuse). Sous-paquets : - `app.py` / `cli.py` : interfaces Streamlit / ligne de commande. - `core/` : `models` (RuleRecord), `scanner`, `classifier`, `rule_registry`, `simulator` (DSL sûr sans eval/exec), `usage_analyzer`, `patcher`, `git_guard`, `test_runner`, `exporters`, `studio` (façade), `pipeline_audit_bridge`. - `extractors/` : `python_ast`, `config`, `markdown`, `comment_rule`, `schema`. - `storage/` : `rule_store` (SQLite), `migrations`. - `agents/` : agents IA (`model_client`, `rule_interpreter_agent`, `rule_coding_agent`, `rule_validation_agent`, `agent_runner`, `prompts`) — l'IA **explique et propose**, les tests **prouvent**, aucun agent n'applique de patch.

11. `scripts/` — scripts d'expérimentation / batch

Fichier	Rôle
<code>scripts/run_extraction_40pages.py</code>	Extraction complète des 40 premières pages de 3 PDF (dump hiérarchie).

scripts/audit_p1_extraction.py	Audit P1 sur de vrais PDF (couverture des champs).
scripts/run_reconstruction.py	Reconstruction depuis les données extraites (coordonnées en points, ordre dessins→images→texte).
scripts/run_reconstruction_validation.py (1449 l.)	Validation de reconstruction (texte attendu, tokens protégés...).
scripts/run_reconstruction_validation_random20.py	Idem sur 20 pages aléatoires.
scripts/run_comparative_analysis.py	Analyse comparative pixel à pixel (RMSE global et par zone 3×3).
scripts/run_background_master_random_40.py	Génère/échantillonne le background master sur 40 pages aléatoires.
scripts/run_object_comprehension_random_audit.py	Audit de compréhension d'objets sur pages aléatoires.
scripts/run_final_page_compiler_random40.py	Lance le compilateur de page finale sur 40 cas.
scripts/run_pageprint_audit.py	Audit PAGEPRINT sur pages réelles.
scripts/run_perfect_reconstruction_experiment.py	Expérience de reconstruction via l'extracteur « parfait ».
scripts/visualize_perfect_extraction_random40.py	Rend des PNG annotés (bbox texte/image/vecteur/table/formule).
scripts/metadata_explorer_builder.py	Construit un explorateur de métadonnées.
scripts/download_visual_layout_models.py	Télécharge les modèles de layout visuel.

12. tests/ — ~125 tests (le contrat exécutable du projet)

Les tests **encodent les règles** que le pipeline ne doit pas casser. Par famille :

Dossier	Nb	Ce qu'ils vérifient
tests/pageprint/	31	Détection (TOC, index, table, légende, figure, en-tête/pied répété, watermark, marque éditeur), préservation des numéros de page/figure, plans de traduction (pas de ligne brute d'index/table/caption traduite, pas de rôle None , pas de token word/char), sanité des titres.
tests/pagereconstruct/	28	⌚ Backends, résolveur de fond, séparation fond/source, expansion de bloc, détecteur de collision, contrat final, fuite de texte source , registre de suppression, survie du texte , pas de double rendu, invariant d'ordre de lecture, dispatcher, similarité de style, gate de publication. À exécuter en premier pour ton debug .
tests/pagetranslate/	22	Couverture de traduction, dédup parent/enfant, verrous de glossaire, préservation des tokens techniques/MLP-CNN, roundtrip des placeholders, projection (bboxes, rôles, tous originaux préservés), fallback batch, raisons de QA.
tests/pubready/	10	Cœur pubready, stages, préservation, intra-bloc, typo (M7), modes M8–M11, audit du cycle de vie du texte source, blockers d'audit visuel d'image.
tests/translation_engines/	13	Batching/décodage CT2, préfixes de langue M2M100/Marian, fallbacks du registre de modèles, mémoire de traduction (hit exact / ignore non validé), variantes de placeholder ASCII/XML, store de profils.
tests/integration/	4	Cas réels de reconstruction, absence d'overlay source (pubready), gates de publication, cas DocIntelligence.
tests/functional/	7	Audit batch (plan vide, rôles manquants, pas de fallback), validateur strict PAGEPRINT, runner d'essai de traduction.
tests/pipelines/	2	Routeur OCR (image-dominant, texte natif insuffisant).
tests/legacy_reconstruction/	1	Contrats legacy.
tests/ (racine, 25)	—	Tests des modules racine : <code>coverage_validator</code> , <code>pageprint</code> , <code>pagetranslate</code> , <code>pipeline_agents</code> , <code>pipeline_orchestrator</code> , <code>reconstructor_font_sizing</code> ⌚ (taille de police — lié à ton bug), <code>background_inpainter</code> , <code>layout_descriptor</code> (v3), <code>llm_semantic_corrector</code> , <code>positioning_policy</code> , <code>publication_qa</code> , etc.

En résumé pour ton debug pagereconstruct/

Les deux pannes que tu décris (texte source non effacé + bbox qui écrase tout sur la 1^{re} ligne) se concentrent dans une poignée de fichiers :

- `layout_box_resolver.py` — le bug « texte redessiné dans la bbox de la 1^{re} ligne » est exactement ce qu'il est censé corriger (`layout_bbox` = bloc complet vs `anchor_bbox` = 1^{re} ligne).
- `patch_planner.py` + `background_resolver.py` + `background_contract.py` — l'effacement du texte source (patches / fond propre).
- `source_text_leak_detector.py` et `pubready/stages/visual_image_audit.py` — la détection que l'effacement a échoué.
- `font_size_sanitizer.py` + le test `test_reconstructor_font_sizing.py` — la taille de police effondrée (texte illisible).

Tests à lancer en priorité : `tests/pagereconstruct/test_source_text_leak.py` , `test_text_survival.py` , `test_layout_and_typography.py` , et `tests/test_reconstructor_font_sizing.py` .